Algorithmic Trading on Energy Spreads: A Data-Driven Implementation in Python

Personal Project - Quantitative Development

February 2026

Executive Summary

This project focuses on the implementation of a systematic trading bot on the WTI/Brent spread. Moving away from theoretical assumptions, the strategy relies on a robust Data Science approach: rigorous data cleaning (handling asynchronous market hours), vectorised backtesting with Pandas, and strict Out-of-Sample validation. The algorithm achieved a Sharpe Ratio of 2.17 on unseen data (2024-2026).

1 Data Engineering & Cleaning

The reliability of a backtest depends on the quality of the data. Retrieving data from `yfinance` required specific processing to align the NYMEX (WTI) and ICE (Brent) calendars.

- **Synchronization:** I implemented an inner-join logic (‘dropna‘) to remove days where one market was closed (e.g., US Labor Day vs UK Bank Holidays), preventing the algorithm from trading on "stale" prices.
- **Feature Engineering:** The Z-Score was calculated on a rolling window to normalize the spread volatility dynamically.

2 Core Algorithm Implementation

The strategy is implemented as a vectorised state machine to ensure execution speed and avoid "look-ahead bias". Below is the core logic used to manage positions, specifically handling the "Hold" state via forward-filling methods.

```
# Step A: Initialize signal container with NaN
data['position'] = np.nan

# Step B: Entry Signals (Z-Score Thresholds)
# Long if Spread is statistically undervalued
data.loc[data['z_score'] < -1.9, 'position'] = 1
# Short if Spread is statistically overvalued
data.loc[data['z_score'] > 1.9, 'position'] = -1

# Step C: Exit Conditions (Mean Reversion)
# We exit when the price returns to equilibrium (|Z| < 0.5)
data.loc[data['z_score'].abs() < 0.5, 'position'] = 0

# Step D: Risk Management (Stop Loss)
# Hard exit if volatility explodes (Z > 3.5)
data.loc[data['z_score'].abs() > 3.5, 'position'] = 0

# Step E: State Propagation (The Logic Fix)
# We propagate the last valid signal forward to maintain positions
data['position'] = data['position'].ffill()

# Step F: Initial State
data['position'] = data['position'].fillna(0)
```

Listing 1: Python Implementation of the Signal Logic

This specific implementation using `ffill()` ensures that the bot stays in position until a clear exit signal (0) is triggered, solving the common issue of "flickering" signals in backtests.

3 Optimization & Robustness

To ensure the model generalizes well to new data:

1. **Avoiding Overfitting:** I ran a Grid Search on 2000-2023 data but deliberately selected the **third-best parameter set** rather than the absolute maximum. This parameter set showed better stability across nearby values.
2. **Out-of-Sample Test:** The model was then run on strictly unseen data (Jan 2024 - Present).

4 Performance Results (Out-of-Sample)

- **Sharpe Ratio:** 2.17
- **Net PnL:** $14.60 per barrel (fees included)

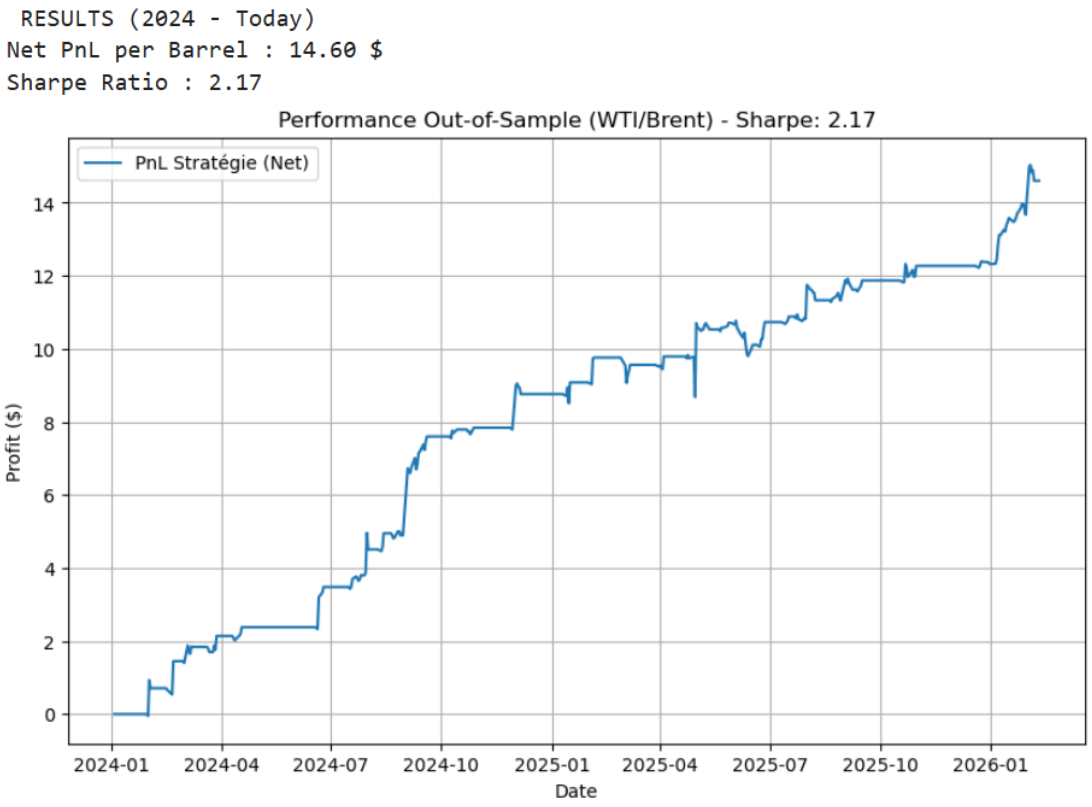

Figure 1: Equity Curve generated by Matplotlib on the test set.

5 Critical Analysis & Real-World Constraints

While the strategy performs well Out-of-Sample, a rigorous quantitative approach requires acknowledging inherent limitations suitable for live deployment:

- **Liquidity & Slippage:** The model assumes a fixed cost of $0.05/bbl. In a live environment, entering positions during high volatility (e.g., OPEC announcements) could incur higher slippage. Future improvements would involve optimizing execution timing (TWAP/VWAP).

- **Static Parameters:** The lookback window ($k = 10$) is constant. A further improvement would be to implement **dynamic parameters** using a Kalman Filter to adjust to changing volatility regimes automatically.
- **Macro Factors:** The model is purely statistical. Integrating exogenous data (inventory levels) could reduce false positives during structural market shifts.

Conclusion: A First Step into Commodity Trading

This project marks my first concrete step into the world of Commodity Trading. It allowed me to move from theoretical concepts to the operational reality of energy markets, specifically the WTI/Brent spread dynamics.

Through this implementation, I confronted two major industry challenges:

- **Data Reality:** Managing the asynchronous calendars of NYMEX and ICE was a practical lesson in the importance of clean data infrastructure before any modeling can occur.
- **Robustness over Hype:** By prioritizing parameter stability (3rd best) over raw historical performance, I adopted a risk-aware approach essential for long-term survival in the markets.

This experience confirms my strong interest in market dynamics and my readiness to deepen these skills within a professional trading desk.

Technology Stack: Python (Pandas, NumPy, Matplotlib), yfinance API.
Code Repository: github.com/adebove/WTI-BRENT-Spread-Trading